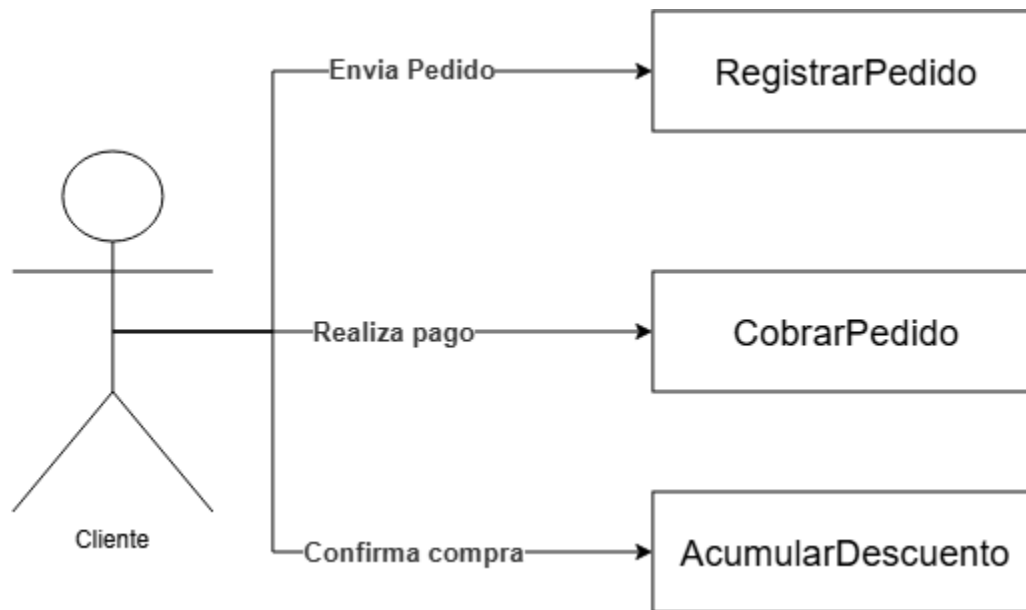


Tarea 1 - Diseño de Servicios SOA

A) Modelado (Top-down)

Servicios identificados:

1. RegistrarPedido — Recibe los pedidos hechos por el cliente en línea y los registra para enviarlos al servicio de cobro.
2. CobrarPedido Recibe el pago de los productos pedidos por el cliente y genera el comprobante virtual del pago realizado.
3. AcumularDescuento Recibe el ticket del pedido y, una vez confirmado el pago, calcula el 2% del total como descuento aplicable a la siguiente compra.



B) Contrato de servicio

Servicio elegido: AcumularDescuento

Operación: calcularDescuento

Entrada:

Campo	Tipo	Obligatorio	Condicion
idPedido	string	si	Debe existir
montoTotal	number	si	Debe ser mayor a 0
idCliente	string	si	Debe existir
pagoConfirmado	boolean	si	Debe ser "true" para procesar

Respuesta:

Campo	Tipo	Descripcion
idDescuento	string	Identificador único del descuento generado
montoDescuento	number	Monto calculado (2% del total)
estatus	string	Valor inicial: "activo"
fechaVigencia	date	Fecha límite de uso (máx. 1 semana)

Errores:

Código HTTP	Significado
400 Bad Request	datos mal formados o incompletos
402 Payment Required	pagoConfirmado llegó en false
404 Not Found	idPedido o idCliente no existen
409 Conflict	ya existe un descuento generado para ese idPedido

Protocolo: REST sobre HTTPS

C) XML Y XSD

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<descuento>
  <idPedido>PED-00125</idPedido>
  <montoTotal>1500.00</montoTotal>
  <idCliente>CLI-0042</idCliente>
  <pagoConfirmado>true</pagoConfirmado>
  <idDescuento>DESC-00078</idDescuento>
  <montoDescuento>30.00</montoDescuento>
  <estatus>activo</estatus>
  <fechaVigencia>2026-08-08</fechaVigencia>
</descuento>
```

XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="descuento">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="idPedido" type="xs:string"/>
        <xs:element name="montoTotal" type="xs:decimal"/>
        <xs:element name="idCliente" type="xs:string"/>
        <xs:element name="pagoConfirmado" type="xs:boolean"/>
        <xs:element name="idDescuento" type="xs:string"/>
        <xs:element name="montoDescuento" type="xs:decimal"/>

        <xs:element name="estatus">
          <xs:simpleType>
            <xs:restriction base="xs:string">
              <xs:enumeration value="activo"/>
              <xs:enumeration value="aplicado"/>
              <xs:enumeration value="vencido"/>
            </xs:restriction>
          </xs:simpleType>
        </xs:element>

        <xs:element name="fechaVigencia" type="xs:date"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Tarea 2

A)

DTO

```
import { IsString, IsNumber, IsBoolean, IsNotEmpty, Min } from 'class-validator';
import { ApiProperty } from '@nestjs/swagger';

export class CalcularDescuentoDto {
  @ApiProperty({ example: 'PED-00125' })
  @IsString()
  @IsNotEmpty()
  idPedido: string;

  @ApiProperty({ example: 1500.00 })
  @IsNumber()
  @Min(0.01)
  montoTotal: number;

  @ApiProperty({ example: 'CLI-0042' })
  @IsString()
  @IsNotEmpty()
  idCliente: string;

  @ApiProperty({ example: true })
  @IsBoolean()
  pagoConfirmado: boolean;
}
```

b) comunicacion asincrono y sincrono

Operacion sincrona: CobrarPedido

Patrón: comunicacion sincrona (REST, respuesta inmediata)

Justificación: el cliente necesita confirmación inmediata de que su pago fue aceptado antes de continuar, no tiene sentido dejarlo en incertidumbre sobre si su dinero se proceso correctamente.

Ventaja: el cliente obtiene una respuesta clara e inmediata sobre el estado de su pago.

Desventaja: si el servicio de pagos esta lento o caido, el cliente queda bloqueado esperando sin poder continuar.

Operacion asincrona: AcumularDescuento

Patron: comunicacion asincrona (procesamiento en segundo plano, ejemplo:cola de mensajes)

Justificacion: el cliente no necesita ver el resultado de este calculo de forma inmediata, puede continuar con su compra o retirarse sin esperar a que el descuento se procese.

Ventaja: no bloquea la experiencia del cliente, mejorando la percepción de la velocidad de la app.

Desventaja: existe una ligera demora entre la compra y que el descuento este realmente disponible para usarse.

C)Orquestacion

Flujo de la saga (3 pasos)

RegistrarPedido- CobrarPedido-AcumularDescuento

Escenario: si falla el ultimo paso (AcumularDescuento)

Si AcumularDescuento falla despues de que RegistrarPedido y CobrarPedido ya se completaron exitosamente (pedido registrado y pago confirmado), la compensación es la siguiente:

- No se cancela el pedido (paso 1).
- No se devuelve el dinero (paso 2)
- Se registra el error del cálculo de descuento.
- Se programa un reintento posteriormente de AcumularDescuento, o se notifica al cliente que su descuento no pudo aplicarse en ese momento.

Justificación:

RegistrarPedido y CobrarPedido son los pasos críticos del proceso del negocio, ya que representan la transacción principal (el cliente recibio su producto y pagó por él).

AcumularDescuento es una operacion secundaria menor de impacto; deshacer una compra Ya completada solo por que fallo el calculo de un beneficio adicional generaria mas friccion e insatisfacción en el cliente de la que resuelve. Por eso, en vez de una compensación destructiva (reversar todo el proceso), se opta por una estrategia de reintento o notificación.

TAREA 3

a) Fórmula de conexiones

Datos:

N=9 sistemas , integrados punto a punto (sin ESB)

Nivel 2 - Integrado.

Fórmula vista en clase

Conexiones= $N(N-1) / 2$

Aplicando formula:

Conexiones = $9(9-1) / 2$

Conexiones = $9(8) / 2$

Conexiones = $72 / 2$

Conexiones = 36

Resultado: 36 conexiones directas entre los 9 sistemas.

B) nivel general

Valores por dimension:

Negocio:6

Organizacion:2

Metodo:2

Aplicacion:1

Arquitectura: 5

Informacion: 3

Infraestructura: 4

Valor más bajo: 1, en la dimensión de Aplicación.

Nivel general: Nivel 1- silos

Justificación: Aunque otras dimensiones muestra mayor madurez (como Negocio en el nivel 6), la regla del eslabon más débil indica que el nivel general del sistema está determinado por su punto más débil. Al estar la dimensión de Aplicación en el nivel 1, el sistema completo se considera en Nivel 1- Silos

Que falta para subir al siguiente nivel:

Para subir del Nivel 1(Silos) al nivel 2 (Integrado) en la dimensión de aplicacion, se necesitaria que las aplicaciones de la cafeteria (registro de pedidos, cobro y descuentos) dejaran de operar de forma aislada y comenzaran a comunicarse automaticamente entre si, por ejemplo: que al completarse RegistrarPedido, esa informacion fluya directamente hacia CobrarPedido, permitiendo que ambas aplicaciones compartan datos de forma conectada.

TAREA 4

A) Renderizado en Next.js

- Página publica (catalogo/menú): SSG (Static Site Generation)

-LCP: Muy bajo, ya que el HTML se genera una sola vez en el build y se sirve estático, sin procesamiento en el servidor al momento de la visita.

-SEO: Óptimo, ya que el contenido completo está presente en el HTML desde la primera carga, facilitando su indexación por buscadores.

Dashboard privado: SSR (Server-Side Rendering)

-LCP: Moderado, mas lento que SSG porque el servidor procesa y consulta datos en cada visita, pero más rápido que CSR porque el HTML llega ya armado con contenido visible.

-SEO: No relevante, ya que es una página privada protegida por login, no indexada por buscadores.

B) Modelo de tenancy

Modelo elegido: Pool

Justificación: Con un presupuesto de infraestructura limitado y la necesidad de atender a cientos de negocios similares, el modelo pool es el más económico, ya que todos los negocios comparten la misma infraestructura (mismo servidor, misma base de datos), evitando el costo de mantener recursos separados para cada cliente.

Riesgos a mitigar desde el diseño:

1.Fuga de datos entre tenants: Al compartir la misma base de datos, existe el riesgo de que, por un error, de filtrado en las consultas, un negocio pueda acceder accidentalmente a informacion de otro negocio (ej. Pedidos, ventas). Debe mitigarse asegurando que cada consulta filtre estrictamente por el identificador del negocio (tenant).

2.Punto unico de falla: si la infraestructura compartida falla o se cae, todos los negocios se ven afectados simultáneamente, a diferencia del modelo Silo, donde un fallo afecta solo a un cliente.

DECLARACION DE IA

Sí utilicé una herramienta de inteligencia artificial (Claude) como apoyo durante el desarrollo de esta actividad, principalmente para comprender y reforzar conceptos relacionados con los temas vistos en clase.

La herramienta me ayudó a:

- Comprender conceptos técnicos necesarios para resolver las tareas, como DTO, contrato de servicio, Saga, OSIMM, tenancy y estrategias de renderizado en [Next.js](#).
- Comprender especialmente los conceptos de **tenancy** y la determinación del **nivel general de OSIMM**, ya que fueron temas que me ayudaron a entender mejor cómo identificar el modelo de aislamiento de los clientes y cómo evaluar el nivel de madurez de la arquitectura, también me ayudo mucho para el XML y XSD, tema que me cuesta un poco en la parte que a veces se me olvida cerrar los XS .
- Aclarar dudas sobre los conceptos y recibir explicaciones que me permitieran comprender mejor su aplicación en mi proyecto.
- Revisar mis propias propuestas y recibir retroalimentación para identificar posibles errores o inconsistencias.
- Comprender el funcionamiento de algunas herramientas técnicas utilizadas durante el desarrollo de la actividad, como draw.io, Lucidchart y Git/GitHub.

Aclaración: En las imágenes que puse de XML, XSD Y DTO, son puestas en carbon code, una herramienta que una vez nos enseñó el profesor allí, para pegar código y luego se convierte en imagen para que se vea mas linda para documentos.

Enlace del Video (tema V4- DTO)

<https://drive.google.com/drive/folders/1MF9S9NTNxKqZ4GkEI0PIKscBN697QCII?usp=sharing>

Enlace de evidencia (repositorio github)

https://github.com/sayuzucha/RECU_SOA

Diagrama editable:

<https://drive.google.com/file/d/1ivFu1LclB9kzS4NKD-dPwB9UJBtAg8mg/view?usp=sharing>